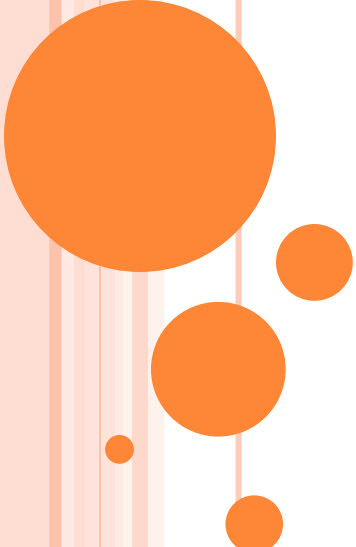




EXEMPLO TDD

Paulyne Jucá (paulyne@ufc.br)

Fonte: livro Test-Driven Development do Mauricio Aniche pela Casa do Código

DESCONTO EM SALÁRIO

Suponha que uma empresa precise calcular o salário do funcionário e seus descontos.

Para calcular esse desconto, a empresa leva em consideração o salário atual e o cargo do funcionário.

Um funcionário possui nome, salário e cargo. O cargo é representado por uma enumeração. Nesse momento, a enumeração contém apenas desenvolvedor, testador e DBA.

As regras de negócio são as seguintes:

- Desenvolvedores possuem 20% de desconto caso seu salário seja maior do que R\$ 3000,0. Caso contrário, o desconto é de 10%.
- DBAs e testadores possuem desconto de 25% se seus salários forem maiores do que R\$ 2500,0. 15%, em caso contrário



VAMOS PENSAR NOS CENÁRIOS DE
TESTE PARA COMEÇAR?

EXEMPLOS DE CENÁRIOS

- ❑ Calcular Salário Para Desenvolvedores Com Salário Abaixo Do Limite
- ❑ Calcular Salário Para Desenvolvedores Com Salário Acima Do Limite
- ❑ Calcular Salário Para DBA Com Salário Abaixo Do Limite
- ❑ Calcular Salário Para Testadores Com Salário Acima Do Limite

*DBA e Testadores fazem parte da mesma classe de equivalência



EM UM PRIMEIRO MOMENTO, QUAIS SERIAM NOSSAS CLASSES?

- ❑ CalculadoraDeSalário (a classe que estamos testando)
- ❑ Funcionário (dependência da classe calculadora)
- ❑ Cargo (enumeração) (dependência da classe calculadora)

*no futuro, vamos aprender a isolar dependências. Por enquanto, vamos implementando as dependências antes para ajudar no desenvolvimento, ok?



CARGO

```
public enum Cargo {  
    DESENVOLVEDOR,  
    DBA,  
    TESTADOR  
}
```



FUNCIONÁRIO

```
public class Funcionario {  
    private String nome;  
    private double salario;  
    private Cargo cargo;  
    public Funcionario(String nome, double salario, Cargo cargo) {  
        this.nome = nome;  
        this.salario = salario;  
        this.cargo = cargo;  
    }  
    public String getNome() {  
        return nome;  
    }  
    public double getSalario() {  
        return salario;  
    }  
    public Cargo getCargo() {  
        return cargo;  
    }  
}
```



PRIMEIRO TESTE

```
public class CalculadoraDeSalarioTest {

    @Test
    public void
    deveCalcularSalarioParaDesenvolvedoresComSalarioAbaixoDoLimite() {

        CalculadoraDeSalario calculadora = new CalculadoraDeSalario();
        Funcionario desenvolvedor = new Funcionario
            ("Mauricio", 1500.0, Cargo.DESENVOLVEDOR);

        double salario = calculadora.calculaSalario(desenvolvedor);

        assertEquals(1500.0 * 0.9, salario, 0.00001);
    }
}
```


CÓDIGO ATÉ AQUI

```
public class CalculadoraDeSalario {  
  
    public double calculaSalario(Funcionario funcionario) {  
        return 1350.0;  
    }  
}
```

Refatore!



SEGUNDO TESTE

```
@Test
public void
deveCalcularSalarioParaDesenvolvedoresComSalarioAcimaDoLimite() {

    CalculadoraDeSalario calculadora = new CalculadoraDeSalario();
    Funcionario desenvolvedor = new Funcionario
        ("Mauricio", 4000.0, Cargo.DESENVOLVEDOR);

    double salario = calculadora.calculaSalario(desenvolvedor);

    assertEquals(4000.0 * 0.8, salario, 0.00001);
}
```



CÓDIGO ATÉ AQUI

```
public double calculaSalario(Funcionario funcionario) {  
    if(funcionario.getSalario() > 3000) return 3200.0;  
    return 1350.0;  
}
```

Refatore!



TERCEIRO TESTE

```
@Test
public void deveCalcularSalarioParaDBAsComSalarioAbaixoDoLimite() {

    CalculadoraDeSalario calculadora = new CalculadoraDeSalario();
    Funcionario desenvolvedor = new Funcionario
        ("Mauricio", 500.0, Cargo.DBA);

    double salario = calculadora.calculaSalario(desenvolvedor);

    assertEquals(500.0 * 0.85, salario, 0.00001);
}
```



CÓDIGO ATÉ AQUI

```
public double calculaSalario(Funcionario funcionario) {  
    if(funcionario.getCargo().equals(Cargo.DESENVOLVEDOR)) {  
        if(funcionario.getSalario() > 3000) return 3200.0;  
        return 1350.0;  
    }  
    return 425.0;  
}
```

Refatore!





**NOSSE CÓDIGO ESTÁ FICANDO FEIO E
COMPLEXO. ISSO VAI ATRAPALHAR A
MANUTENÇÃO FUTURA, CERTO?**

**Escolhemos a codificação mais simples, mas não a
modificação (evolução/manutenção) mais simples**



O QUE FIZEMOS ATÉ AGORA NOS AJUDOU
A ENTENDER O PROBLEMA.
MAS É HORA DE ORGANIZAR NOSSO
PROJETO

CÓDIGO ATÉ AQUI

```
public double calculaSalario(Funcionario funcionario) {  
    if(funcionario.getCargo().equals(Cargo.DESENVOLVEDOR)) {  
        if(funcionario.getSalario() > 3000) {  
            return funcionario.getSalario() * 0.8;  
        }  
        return funcionario.getSalario() * 0.9;  
    }  
    else if(funcionario.getCargo().equals(Cargo.DBA)) {  
        funcionario.getCargo().equals(Cargo.TESTADOR)) {  
            if(funcionario.getSalario() < 2500) {  
                return funcionario.getSalario() * 0.85;  
            }  
            return funcionario.getSalario() * 0.75;  
        }  
    }  
  
    throw new RuntimeException("Funcionario invalido");  
}
```

extrair

extrair



CÓDIGO ATÉ AQUI

```
public class CalculadoraDeSalario {

    public double calculaSalario(Funcionario funcionario) {
        if(funcionario.getCargo().equals(Cargo.DESENVOLVEDOR)) {
            return dezOuVintePorCentoDeDesconto(funcionario);
        }
        else if(funcionario.getCargo().equals(Cargo.DBA) ||
funcionario.getCargo().equals(Cargo.TESTADOR)) {
            return quinzeOuVinteCincoPorCentoDeDesconto(funcionario);
        }

        throw new RuntimeException("Funcionario invalido");
    }

    private double
    quinzeOuVinteCincoPorCentoDeDesconto(Funcionario funcionario) {
        if(funcionario.getSalario() < 2500) {
            return funcionario.getSalario() * 0.85;
        }
        return funcionario.getSalario() * 0.75;
    }
}
```





**MAS ISSO É O SUFICIENTE PARA
SIMPLIFICAR NOSSO CÓDIGO?**

Nossa bateria de testes está coesa?

OUTRO PROBLEMA: MÉTODOS PRIVADOS

```
public class CalculadoraDeSalario {  
  
    public double calculaSalario(Funcionario funcionario) {  
        if(funcionario.getCargo().equals(Cargo.DESENVOLVEDOR)) {  
            return dezOuVintePorCentoDeDesconto(funcionario);  
        }  
        else if(funcionario.getCargo().equals(Cargo.DBA) ||  
funcionario.getCargo().equals(Cargo.TESTADOR)) {  
            return quinzeOuVinteCincoPorCentoDeDesconto(funcionario);  
        }  
  
        throw new RuntimeException("Funcionario invalido");  
    }  
  
    private double  
    quinzeOuVinteCincoPorCentoDeDesconto(Funcionario funcionario) {  
        if(funcionario.getSalario() < 2500) {  
            return funcionario.getSalario() * 0.85;  
        }  
        return funcionario.getSalario() * 0.75;  
    }  
}
```



COMO RESOLVE?

```
public interface RegraDeCalculo {  
    double calcula(Funcionario f);  
}
```

```
public class DezOuVintePorCento implements RegraDeCalculo {
```

```
public class QuinzeOuVinteCincoPorCento implements RegraDeCalculo {
```



Novo CÓDIGO PARA CARGO

```
public enum Cargo {  
    DESENVOLVEDOR(new DezOuVintePorCento()),  
    DBA(new QuinzeOuVinteCincoPorCento()),  
    TESTADOR(new QuinzeOuVinteCincoPorCento());  
  
    private final RegraDeCalculo regra;  
  
    Cargo(RegraDeCalculo regra) {  
        this.regra = regra;  
    }  
  
    public RegraDeCalculo getRegra() {  
        return regra;  
    }  
}
```



Novo CÓDIGO PARA CALCULADORA

```
public class CalculadoraDeSalario {  
    public double calculaSalario(Funcionario funcionario) {  
        return funcionario.getCargo().getRegra().calcula(funcionario);  
    }  
}
```



COESÃO NOS TESTES – O QUE MUDA?

- ❑ Os testes também ficam mais coesos pois ficam agrupados apenas quando tratam da mesma regra de cálculo.
- ❑ Antes:
 - Um teste para cada classe, mesmo que a regra de negócio seja igual -> erro na abstração (polimorfismo)
 - Número crescente de testes

```
deveCalcularSalarioParaDesenvolvedoresComSalarioAbaixoDoLimite()  
deveCalcularSalarioParaDesenvolvedoresComSalarioAcimaDoLimite()  
deveCalcularSalarioParaDBAsComSalarioAbaixoDoLimite()  
deveCalcularSalarioParaDBAsComSalarioAcimaDoLimite()  
deveCalcularSalarioParaTestadoresComSalarioAbaixoDoLimite()  
deveCalcularSalarioParaTestadoresComSalarioAcimaDoLimite()
```

COESÃO NOS TESTES – O QUE MUDA?

□ Depois

- deveCalcularSalarioComDescontoDezPorCentoAbaixoDeTresMil
- deveCalcularSalarioComDescontoDezPorCentoAcimaDeTresMil
- deveCalcularSalarioComDescontoQuinzePorCentoAbaixoDeDoisMilEQuinhentos
- deveCalcularSalarioComDescontoVinteECincoPorCentoAcimaDeDoisMilEQuinhentos



COESÃO NOS TESTES – O QUE AVALIAR?

- ❑ Muitos testes para um método
- ❑ Muitos testes para uma classe
- ❑ Cenário muito grande e que lidam com muitos objetos ou fazem muita coisa
- ❑ Teste de método não público
 - Se é importante testar, deve ser uma classe separada
- ❑ Classes devem ser simples

